

Prelude

Please submit your solution using:

`handin cs-418 hw2` Your solution should contain three files:

`hw2.erl`: Erlang source code for your solutions to the questions.

`hw2_tests.erl`: Erlang source code for your solutions to the questions.

`hw2.pdf`: Answers to written questions.

A templates for [hw2.erl](http://www.students.cs.ubc.ca/~cs-418/2024-2/hw/2/hw2.html) and other wonderful goodies are available at

<http://www.students.cs.ubc.ca/~cs-418/2024-2/hw/2/hw2.html>
for your coding convenience.

Please submit code that compiles without errors or warnings. If your code does not compile, we might give you zero points on all of the programming problems. If we fix your code to make it compile, we will take off lots of points for that service. If your code generates compiler warnings, we will take off points for that as well, but not as many as for code that doesn't compile successfully. Any question about whether or not your code compiled as submitted will be determined by trying it on CS department linux machines.

We will take off points for code that prints results unless we specifically asked for print-out. For this assignment, the functions you write should return the specified values, but they should not print anything to `stdout`. Using [lists:format](#) when debugging is great, but you need to delete or comment-out such calls before submitting your solution. Your code must fail with some kind of error when called with invalid arguments.

1 The Questions (65 points)

1. Scan by the book. (20 points)

In the [September 17](#) lecture, we implemented `reduce` using the bottom-up approach presented in the Lin & Snyder text. The template file, `hw2.erl` has the `create/2` and `reduce/3` functions from that lecture. For this question, you will implement `scan`.

- (a) (10 points) Implement `scan(ProcInfo, CombineFun, AccIn, Value)` where `scan` is called by the worker processes:

`ProcInfo` is the `ProcInfo` for the process calling `scan`;

`CombineFun(Left, Right)` is the function for combining values from the left and right subtrees of the current node.

`AccIn` is the initial value for the accumulation. All workers should call `scan` with the same value for `AccIn`.

`Value` is the value for the scan for this worker.

`scan` implements a “exclusive” scan: the value returned by `scan` is the value obtained by combining all values prior to the current worker. In other words, it doesn't include the value from the current worker. For example, if

`CombineFun(Left, Right) -> Left + Right end,`

and we have 8 workers processes whose values for the scan are the elements of

`[1,2,3,4,5,6,7,8],`

then the values returned by `scan(ProcInfo, CombineFun, AccIn, Value)` will be `[0,1,3,6,10,15,21,28]`.

Note that you can get the effect of an inclusive scan that includes the value from the current worker by applying

`CombineFun(ValueFromScan, Value)`

We can't necessarily get the value for an exclusive scan from an inclusive scan; that only works if `CombineFun` is invertible.

See the test case in `hw2_tests.erl` for an executable example.

My implementation uses a `scan/5` that is based on the code from `hw2:reduce/4` with an extra parameter for `AccIn`. You can solve this problem "easily" by copying the code from `reduce/4`, adding a `AccIn` parameter, and making the changes to convert a reduce to a scan. Then, make `scan/4` call your `scan/5` just like `reduce/3` calls `reduce/4`. You can probably find a harder way to solve the problem, but we may take off points if you make the code excessively complicated.

- (b) **(5 points)** Add a test case to `hw2_tests.erl`. Use a function for `CombineFun` that is both associative and commutative and not a test-case example from `hw2_tests.erl`. Clearly state in your comments that this is a test case for a function that is associative and commutative.
- (c) **(5 points)** Add a test case to `hw2_tests.erl`. Use a function for `CombineFun` that is associative but not commutative and is not a test-case example from `hw2_tests.erl`. Clearly state in your comments that this is a test case for a function that is associative but not commutative.

2. Help find my moose. **(25 points)**

The moose that was in my refrigerator for [PIKA 2](#) has escaped! Please help me find the moose.

In a bit more detail, let `WorkerTree` be a tree of processes created by `wtree:create/1`. Let `Key` be the key for a list that is distributed across the workers of `WorkerTree`. In other words,

`lists:append(wtree:retrieve(WorkerTree, Key))`

returns this list. How many times does the string "moose" occur in this list?

The function `hw2:moose_ref(WorkerTree, Key)` is a sequential implementation that retrieves the list associated with `Key` as described above and returns the number of occurrences of "moose" in that list. For this problem, you will write a parallel version using `wtree:reduce`.

- (a) **(5 points)** Include in the `hw2.erl` file that you submit, a comment (probably 5 to 10 lines) a high level description of your implementation:
 - Which of the patterns for reduce from the [September 13 lecture](#) does your design use? If your design is a different pattern, please explain.
 - Describe the "type" of the value returned by your `Leaf` function, and used as the arguments and return type of your `Combine` function. For example, if it's a list, say that it's a list and describe the type of the elements. If it's a tuple, say it's a tuple, and describe the types of the elements.
 - Is there anything else another programmer would want to know to understand your implementation.

This description should be a fairly high-level description of the design. Don't write a line-by-line English approximation of what your code does; you may assume that we can read Erlang code.

- (b) **(5 points)** Implement the leaf function, `hw2:moose_leaf(ProcState, Key)`.
- (c) **(5 points)** Implement the combine function, `hw2:moose_combine(Left, Right)`.
- (d) **(5 points)** Implement the top-level function, `hw2:moose_par(WorkerTree, Key)`. If `WorkerTree` is a worker tree, and `Key` is a valid key for a list distributed across the workers of `WorkerTree`, then

`hw2:moose_par(WorkerTree, Key) == hw2:moose_seq(WorkerTree, Key)`

should evaluate to `true`.

- (e) (5 points) Add at least three test cases to each of `hw2_tests:moose_leaf_test()`, `hw2_tests:moose_combine_test()`, and `hw2_tests:moose_par_test()`.

Notes:

- Be careful about cases where the string "moose" is spread across two or more workers. Reading test cases provided in `moose_par_test` in `hw2_tests.erl` may be helpful for giving you ideas of some of the cases to consider. See also `moose:data` in `moose.erl`.
- I was going to add a question to measure the speed-up of `moose_par` wrt. `moose_seq` on `thetis.cs.ubc.ca`. From my measurements, the results are disappointing. I might do a some more forensic work on that and include an analysis in the solution set. In the meantime, I'll put the speed-up measurement question into the next problem.

3. Queues. (~20 points)
Coming soon!

2 Why?

Question 1: Implementing scan is a good exercise to take the “magic” out of the algorithm.

Note: it was brought to my attention that the file names for my PDF uploads to Canvas could be more descriptive. One is Chapter 5 of the Lin & Snyder book. The other is my introduction to their Peril-L notation. If you are finding their notation to be cryptic and you haven't read my introduction, you probably want to download that file as well.

Question 2: An example of the {Left, Overall, Right} pattern for reduce and scan.

Question 3: Monte Carlo simulations, i.e. randomized simulations are very widely used. They are also an example of an “embarrassingly parallel” problem. Because each of the simulation runs is supposed to be independent of the others, many processors can work each on their own set of simulations. We need to combine these results at the end, typically creating some kind of summary. This combination of the results is often an example of reduce, it is definitely is a reduce for the problem posed here. Simulating queues will help us better understand the routing networks used in messaging passing machines.



Unless otherwise noted or cited, the questions and other material in this homework problem set is copyright 2025 by Mark Greenstreet and are made available under the terms of the Creative Commons Attribution 4.0 International license <http://creativecommons.org/licenses/by/4.0/>